المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⴰⵍ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Rapport des travaux pratiques

Architecture Monolithique, Monolithe Modulaire, Architecture Distribuée et Architecture SOA

Filière : Intelligence Artificielle et Cybersécurité

Module :
Architecture Logicielle

Réalisé par :

Abderrahman MACHHOURI

Encadré par :
Pr. BE EL BAGHAZAOUI

Année universitaire : 2025-2026

Table des matières

Liste des figures **2**

Liste des tableaux **3**

1 TP3 : Architecture Distribuée avec Node.js et Docker **4**

1.1 Objectifs 4
1.2 Architecture cible du système 4
1.3 Lien vers le code 4
1.4 Structure du projet 5
1.5 Flux de données 5
1.6 Lancement du système complet 5
1.6.1 Démarrer tous les conteneurs 5
1.6.2 Vérifier que tout est actif 6
1.7 Tests avec Postman 9
1.7.1 Test 1 : Créer une commande (POST) 9
1.7.2 Test 2 : Lister les commandes (GET) 9
1.7.3 Test 3 : Load balancing 11
1.8 Questions de compte-rendu 13

Table des figures

1.1 Lancement de tous les conteneurs avec Docker Compose 6
1.2 Vérification Nginx + Node.js via localhost/health 7
1.3 Vérification directe de l'Instance 1 (port 3001) 7
1.4 Vérification directe de l'Instance 2 (port 3002) 8
1.5 Interface web de gestion RabbitMQ 8
1.6 POST - Créer une commande (réponse 201 Created) 9
1.7 GET - Premier appel, données servies depuis MongoDB 10
1.8 GET - Deuxième appel, données servies depuis le cache Redis 10
1.9 Load Balancing - Requête 1 traitée par Instance-1 11
1.10 Load Balancing - Requête 2 traitée par Instance-2 11
1.11 Load Balancing - Requête 3 traitée par Instance-1 12
1.12 Load Balancing - Requête 4 traitée par Instance-2 12

Liste des tableaux

1.1 Composants de l'architecture distribuée 4

TP3 : Architecture Distribuée avec Node.js et Docker

1.1 Objectifs

- Déployer une architecture distribuée complète avec plusieurs composants interconnectés.
- Comprendre le rôle de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages.
- Implémenter un flux complet : POST commande → stockage MongoDB → cache Redis → notification RabbitMQ.
- Tester et observer le comportement distribué avec Postman.

1.2 Architecture cible du système

Table 1.1 – Composants de l'architecture distribuée

Composant	**Technologie**	**Rôle**
Client	Postman	Envoie les requêtes HTTP POST/GET pour créer et lister des commandes
Load Balancer	Nginx	Reçoit le trafic et le répartit entre 2 instances Node.js (port 3001 et 3002)
Serveur applicatif	Node.js / Express	Implémente la logique métier : validation, traitement et réponse
Base de données	MongoDB	Stocke toutes les commandes de façon persistante
Cache	Redis	Mémorise les résultats GET pour éviter des requêtes répétitives à MongoDB
Bus de messages	RabbitMQ	Reçoit un événement après chaque commande et affiche une notification

1.3 Lien vers le code

Le code source de cette architecture est disponible sur GitHub :
https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/TP3_architecture_distribuee

1.4 Structure du projet

Le projet suit une architecture microservices orchestrée avec Docker Compose. Voici l'arborescence des fichiers principaux :

```
tp-archi-distribuee/
 docker-compose.yml        # orchestre tous les conteneurs
 nginx/
    nginx.conf            # configuration du load balancer
 app/
    package.json
    server.js             # serveur Node.js principal
    routes/
       commandes.js      # routes API
    services/
       mongodb.js        # connexion MongoDB
       redis.js          # connexion Redis
       rabbitmq.js       # connexion RabbitMQ
    consumer.js           # consommateur de messages
```

1.5 Flux de données

Postman → Nginx :80 → Node.js :3001 / :3002
Node.js → MongoDB (stockage persistant)
Node.js → Redis (cache GET /commands)
Node.js → RabbitMQ → Consommateur → Notification console

1.6 Lancement du système complet

1.6.1 Démarrer tous les conteneurs

Depuis la racine du projet, exécuter la commande suivante :

```
docker-compose up --build
```

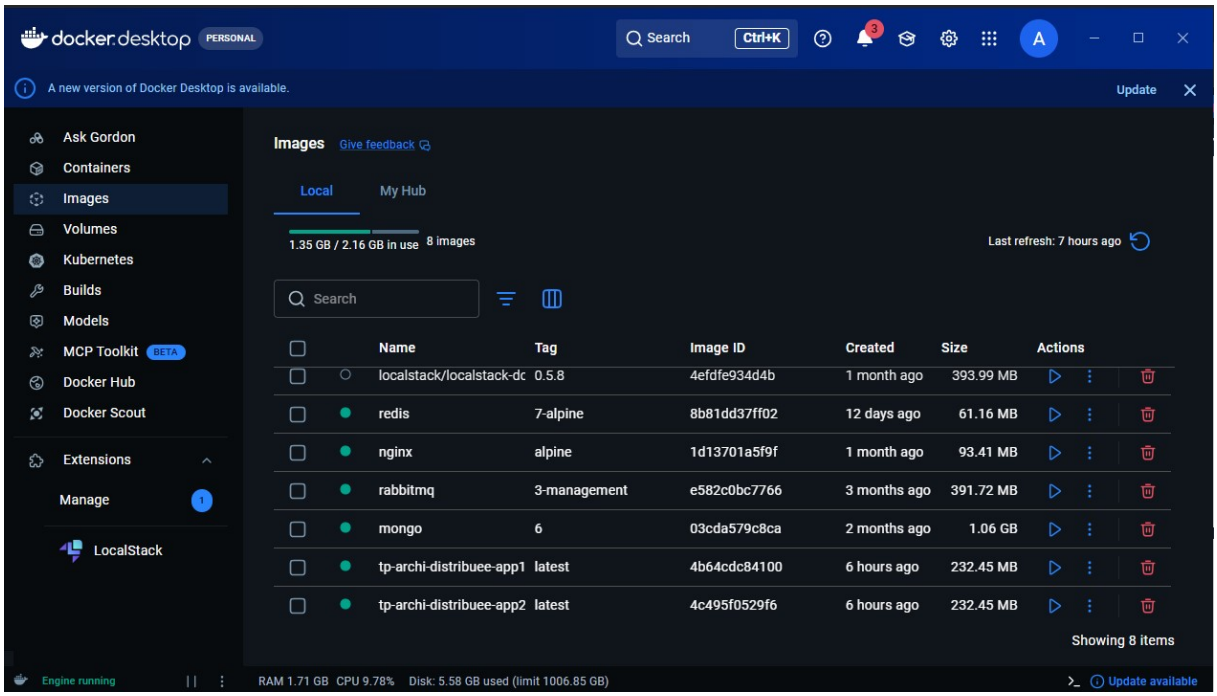

FIGURE 1.1 – Lancement de tous les conteneurs avec Docker Compose

1.6.2 Vérifier que tout est actif

Vérification Nginx + Node.js

URL : http://localhost/health
Résultat attendu :

```
{ status: 'OK', instance: 'Instance-1' }
```

ou

```
{ status: 'OK', instance: 'Instance-2' }
```

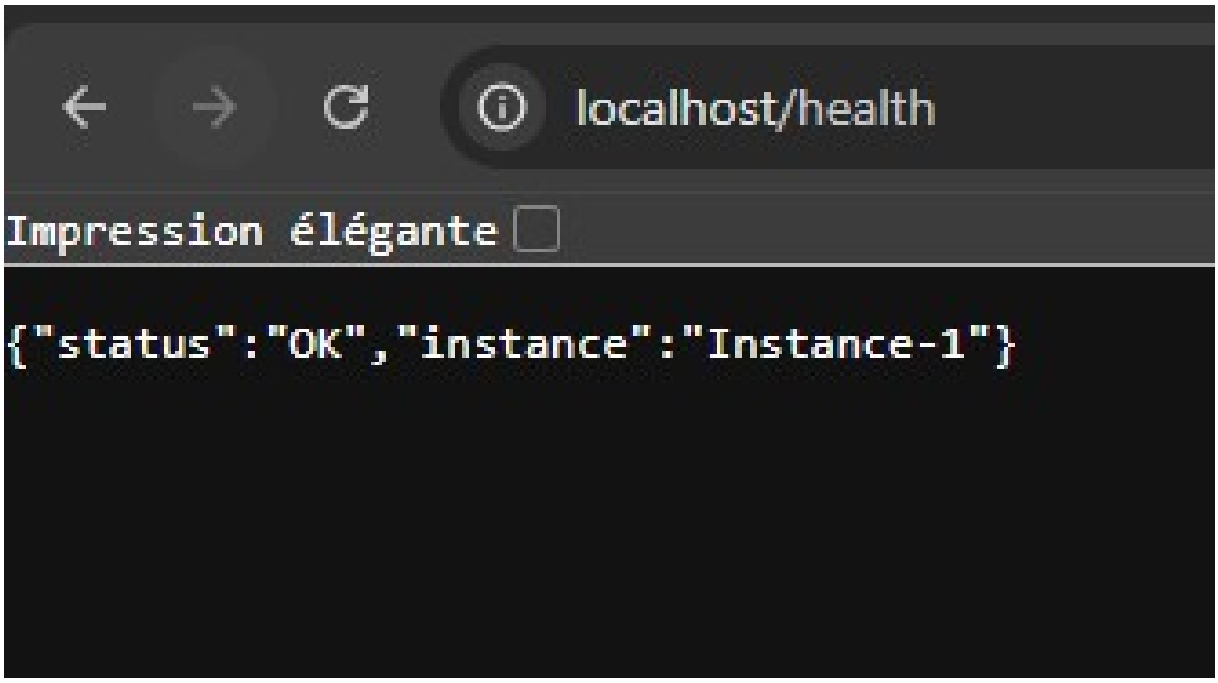

FIGURE 1.2 – Vérification Nginx + Node.js via localhost/health

Vérification Instance 1 directe

URL : http://localhost:3001/health
Résultat attendu :

```
{ status: 'OK', instance: 'Instance-1' }
```

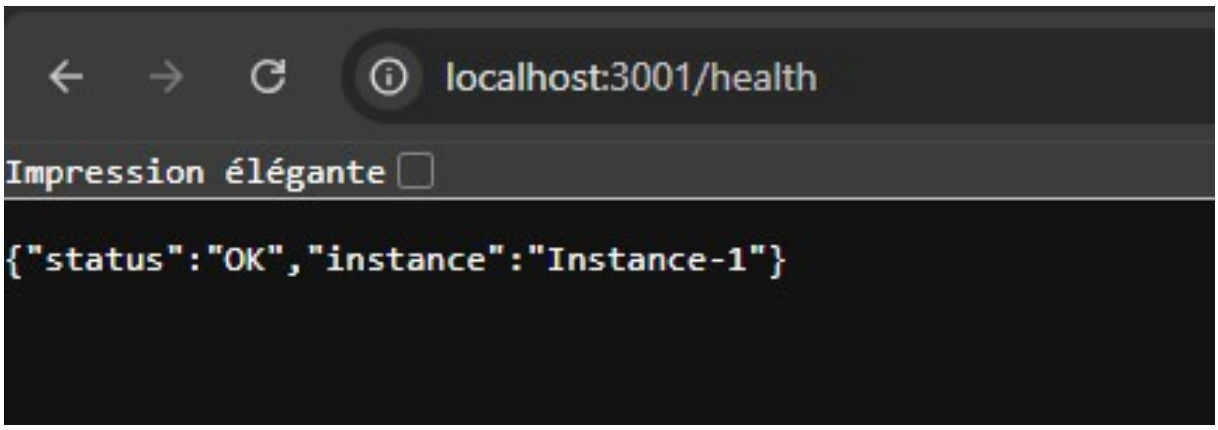

FIGURE 1.3 – Vérification directe de l'Instance 1 (port 3001)

Vérification Instance 2 directe

URL : http://localhost:3002/health
Résultat attendu :

```
{ status: 'OK', instance: 'Instance-2' }
```

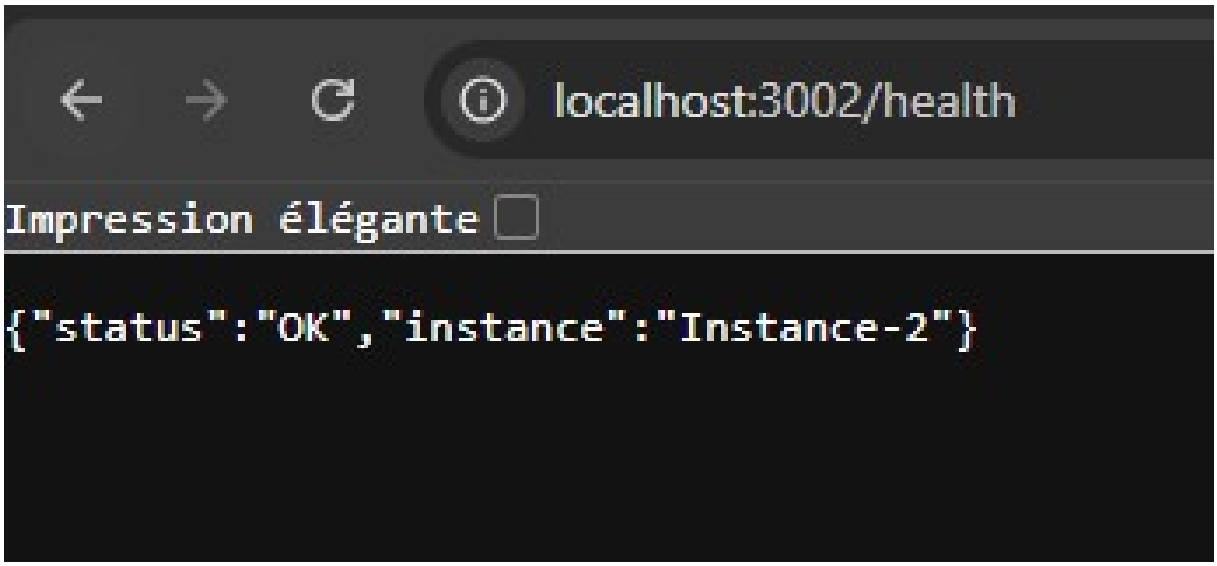

FIGURE 1.4 – Vérification directe de l'Instance 2 (port 3002)

Vérification RabbitMQ

URL : http://localhost:15672
Identifiants : admin / admin
Résultat attendu : Interface web de gestion RabbitMQ

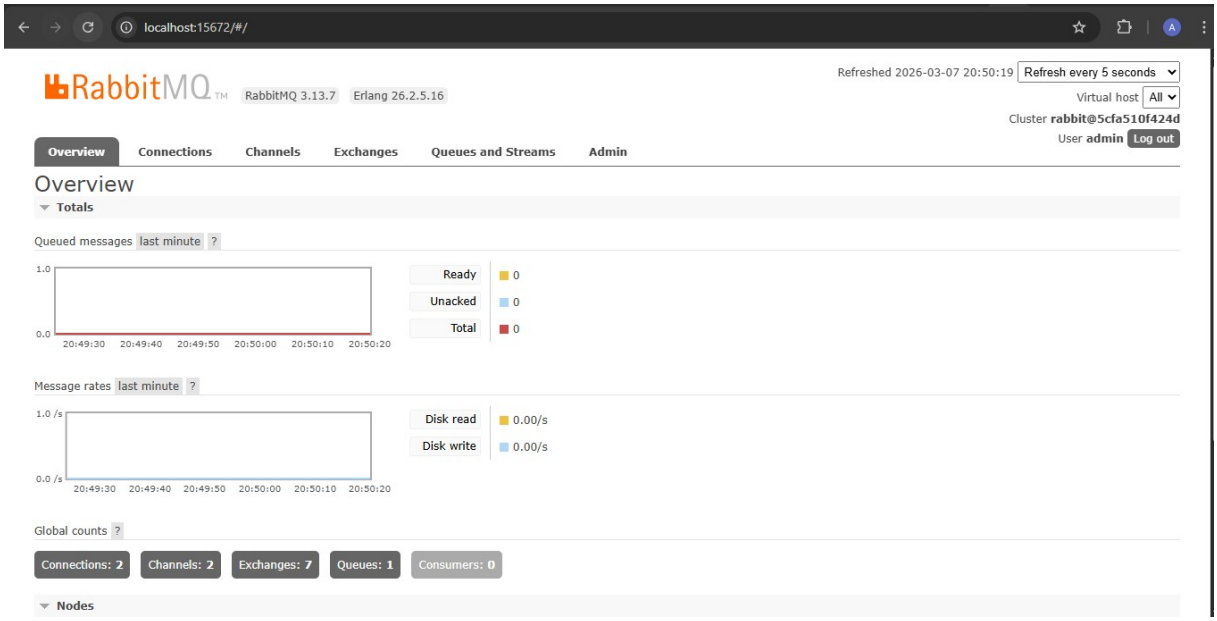

FIGURE 1.5 – Interface web de gestion RabbitMQ

1.7 Tests avec Postman

1.7.1 Test 1 : Créer une commande (POST)

— Méthode : POST
— URL : `http://localhost/commands`
— Headers : Content-Type : application/json
— Body :

```
{
  "produit": "Laptop Dell XPS",
  "quantite": 2,
  "client": "Ahmed Benali"
}
```

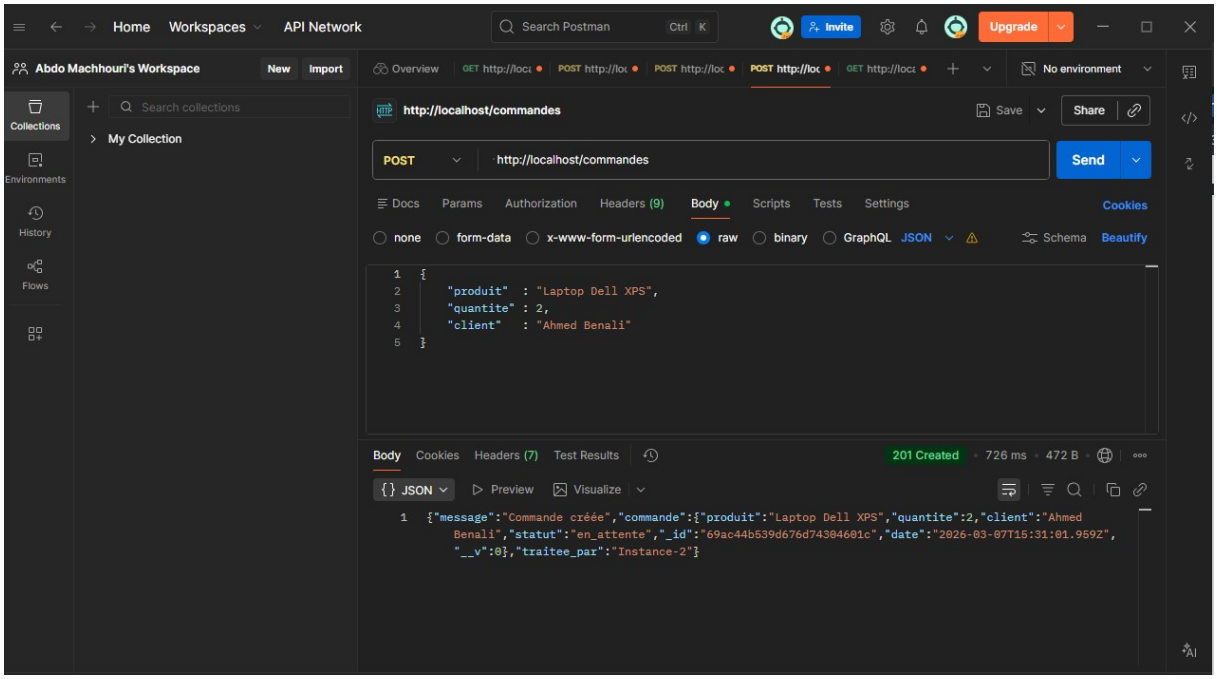

FIGURE 1.6 – POST - Créer une commande (réponse 201 Created)

Le serveur répond avec un code 201 Created, ce qui indique que la commande a été créée avec succès.

1.7.2 Test 2 : Lister les commandes (GET)

— Méthode : GET
— URL : `http://localhost/commands`
— 1er appel → source MongoDB :

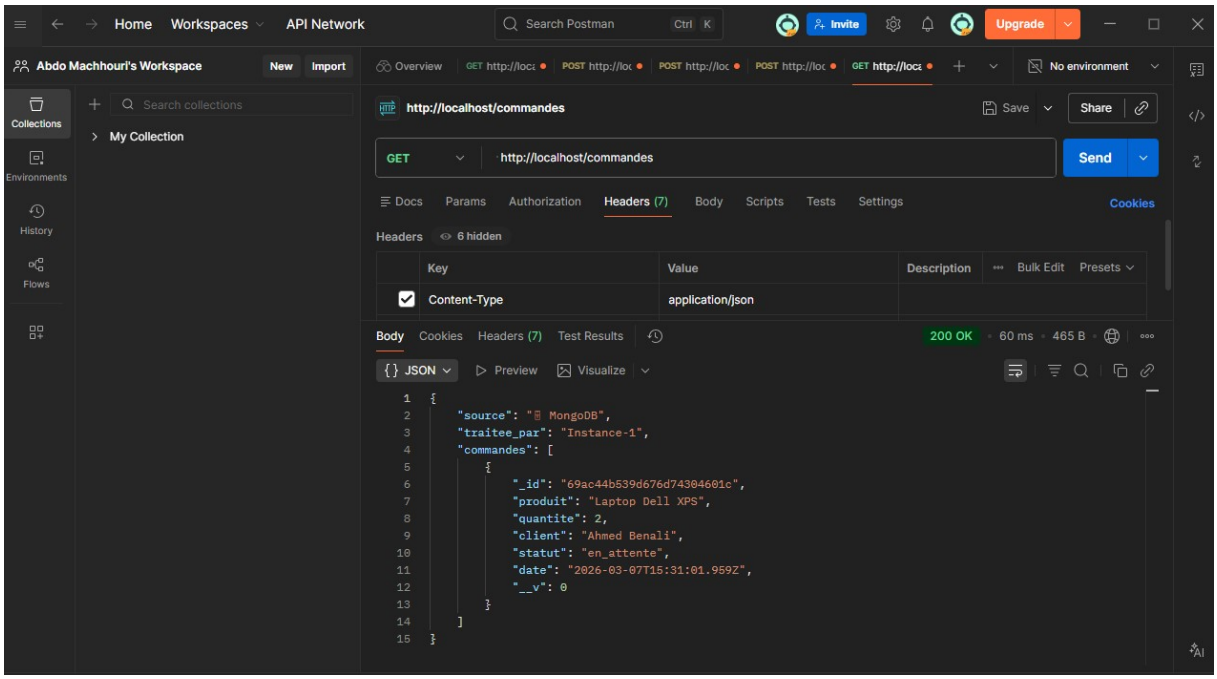

FIGURE 1.7 – GET - Premier appel, données servies depuis MongoDB

La présence du champ `"source": "MongoDB"` confirme que les données proviennent bien de la base MongoDB.

— 2ème appel → source Redis (cache) :

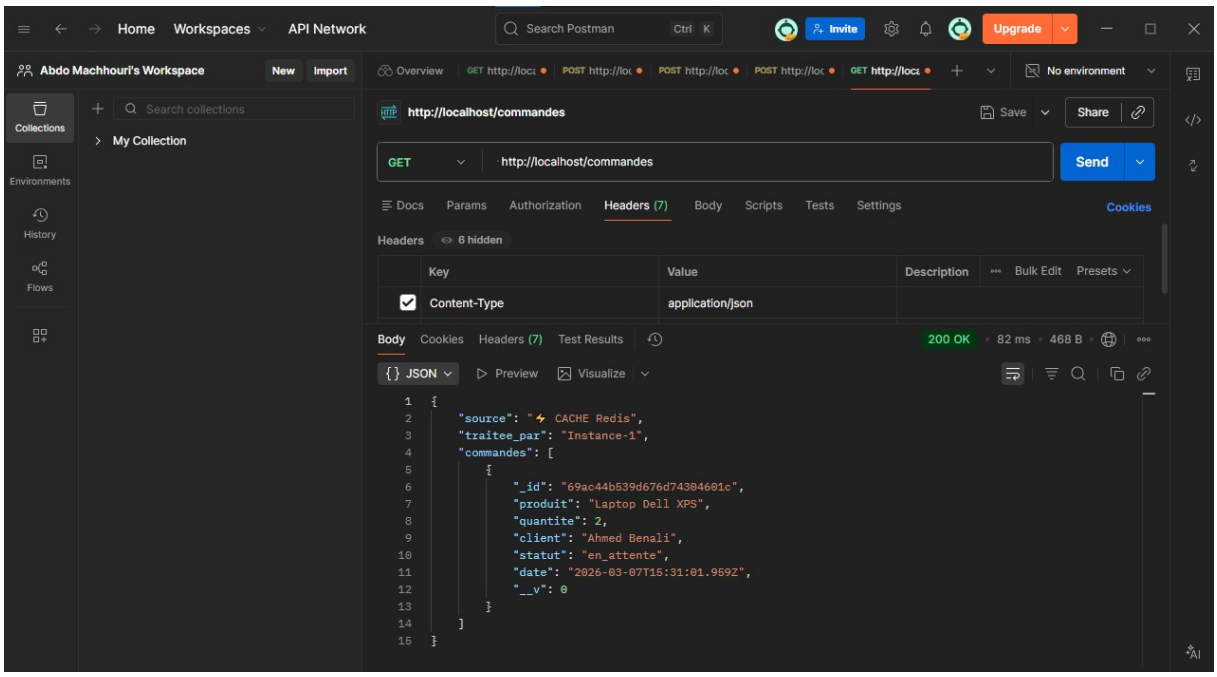

FIGURE 1.8 – GET - Deuxième appel, données servies depuis le cache Redis

Le champ `"source": "CACHE Redis"` confirme que les données ont été servies depuis le cache Redis.

1.7.3 Test 3 : Load balancing

Envoyer 4 requêtes POST successives et observer le champ `traitee_par` :

— Requête 1 → traitee_par : Instance-1

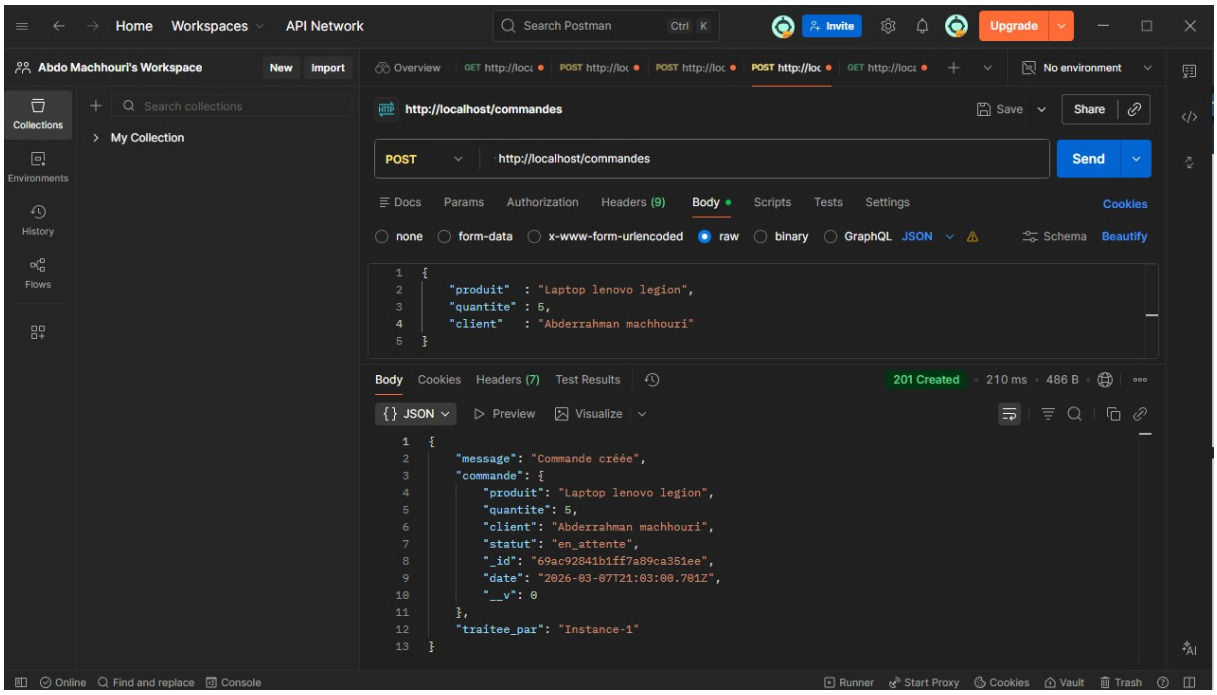

FIGURE 1.9 – Load Balancing - Requête 1 traitée par Instance-1

— Requête 2 → traitee_par : Instance-2

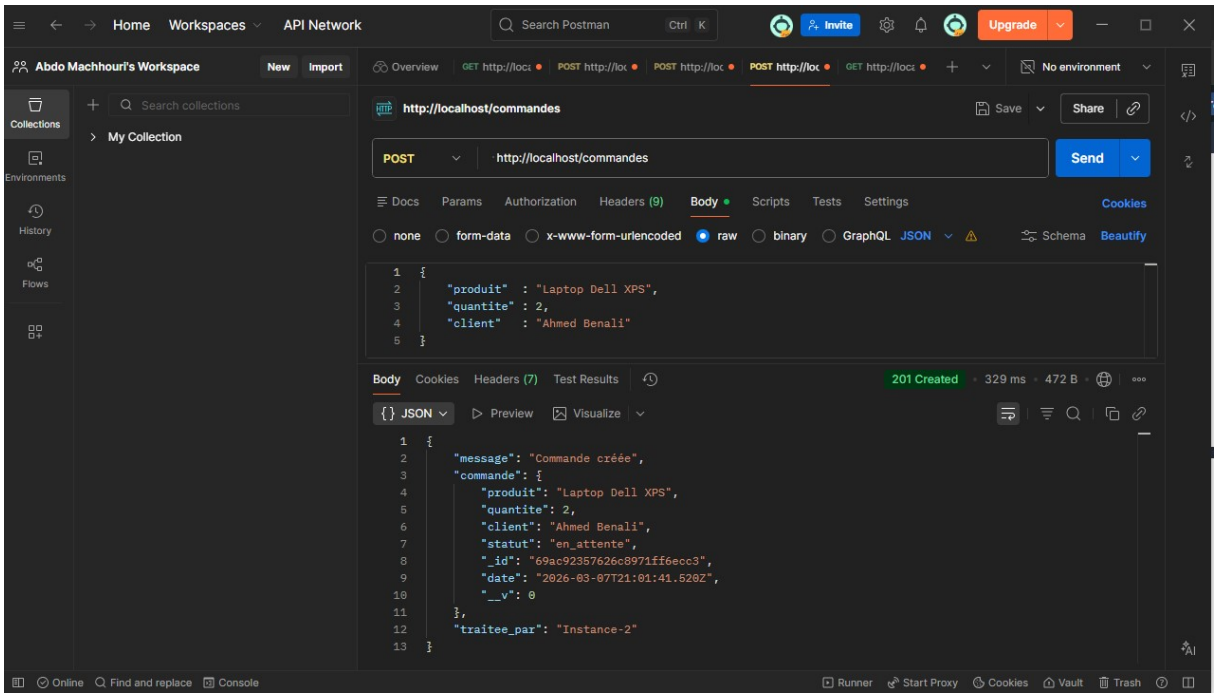

FIGURE 1.10 – Load Balancing - Requête 2 traitée par Instance-2

— Requête 3 → traitee_par : Instance-1

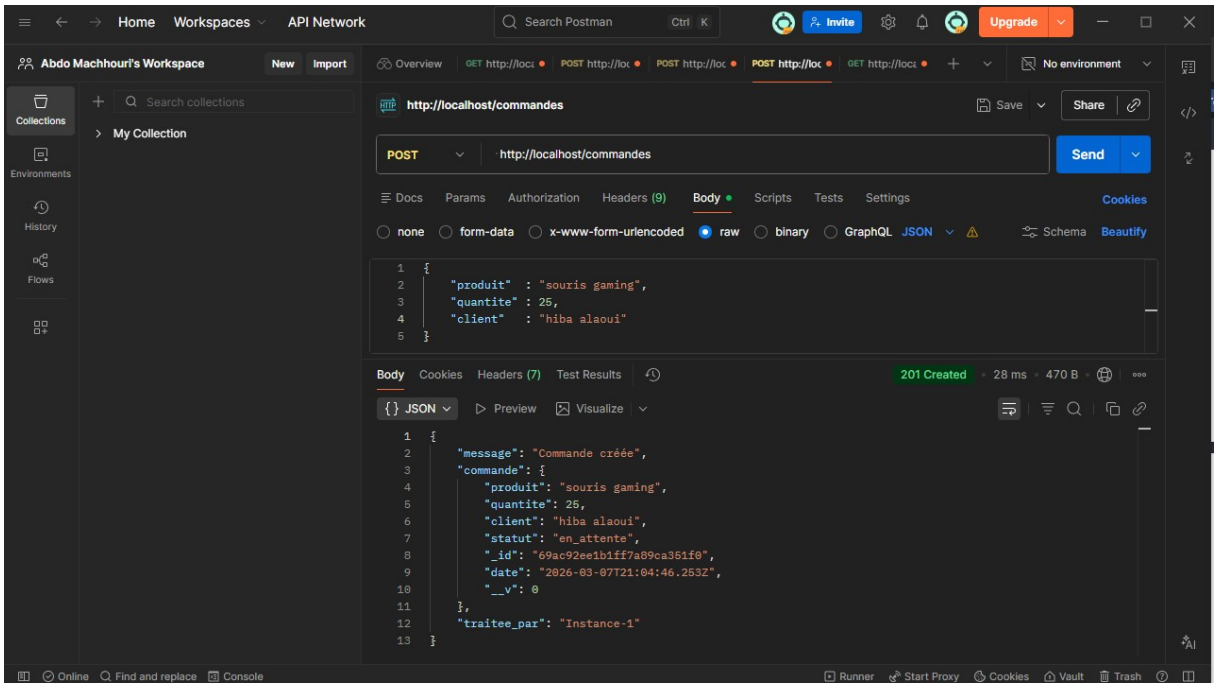

FIGURE 1.11 – Load Balancing - Requête 3 traitée par Instance-1

— Requête 4 → traitee_par : Instance-2

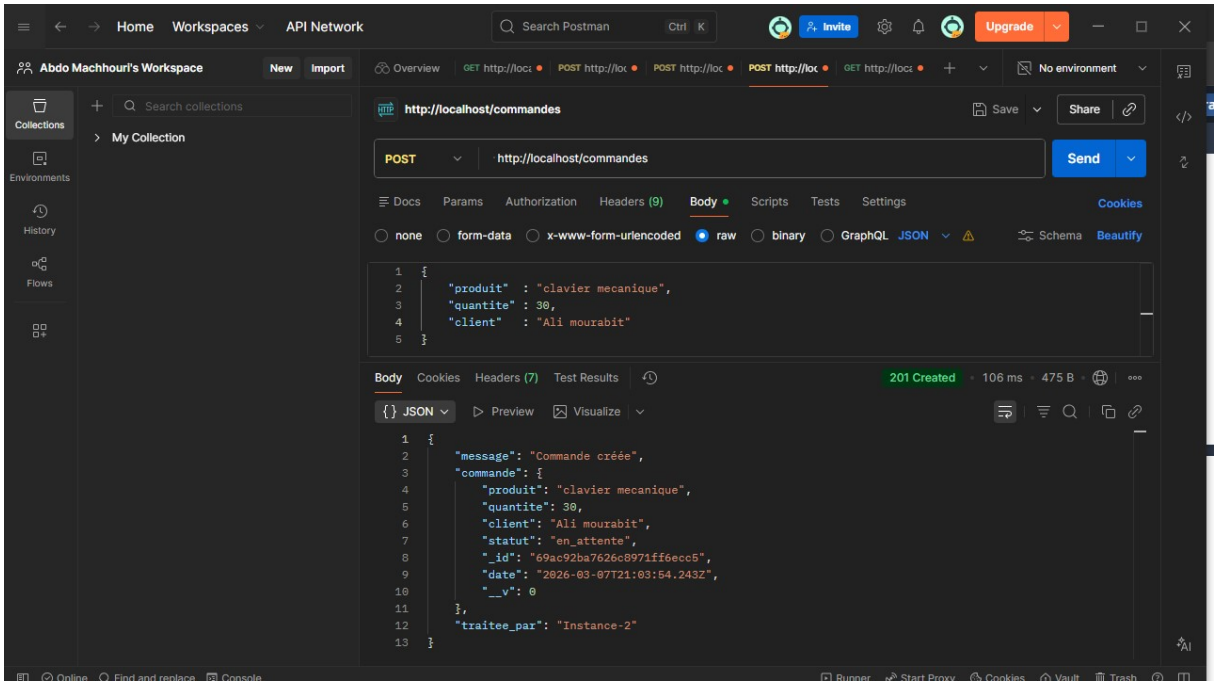

FIGURE 1.12 – Load Balancing - Requête 4 traitée par Instance-2

Cela confirme que Nginx répartit la charge en Round-Robin.

1.8 Questions de compte-rendu

1. **Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?**
   Le composant qui reçoit en premier les requêtes est **Nginx (Load Balancer)**. Il utilise l'algorithme **Round-Robin** par défaut, qui distribue les requêtes de manière séquentielle et équitable entre les instances Node.js (app1 et app2), en alternant à chaque nouvelle requête.
2. **Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?**
   Si app1 tombe en panne, Nginx continue de fonctionner et redirige automatiquement toutes les requêtes vers app2. Cela est possible grâce au mécanisme de **health check passif** de Nginx, qui détecte les serveurs indisponibles et les exclut temporairement de la rotation, assurant ainsi la continuité du service.
3. **Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET (source Redis).**
   Le **1er GET** interroge directement la base de données **MongoDB**, ce qui implique une lecture sur disque et donc un temps de réponse plus élevé. Le **2ème GET** est servi depuis le cache **Redis**, qui stocke les résultats en mémoire vive. Redis étant beaucoup plus rapide que MongoDB, le temps de réponse est considérablement réduit, ce qui allège également la charge sur la base de données.
4. **Pourquoi invalide-t-on le cache Redis après un POST /commands ?**
   Le cache Redis est invalidé après un **POST /commands** afin de maintenir la **cohérence des données**. En effet, si le cache n'était pas invalidé, les prochaines lectures retourneraient des données obsolètes ne reflétant pas les nouvelles entrées insérées dans MongoDB. L'invalidation force donc une nouvelle lecture depuis MongoDB lors du prochain GET.
5. **En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ?**
   RabbitMQ améliore la résilience du système en permettant un traitement **asynchrone** des événements. Les messages sont publiés dans une file d'attente et persistent même si le consommateur est temporairement indisponible. Lorsque le consommateur redémarre, il reprend le traitement des messages sans perte de données, garantissant ainsi la **fiabilité** et la **non-perte d'événements** dans le système.
6. **Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ?**
   Il faut ajouter une nouvelle ligne dans le bloc `upstream backend` du fichier `nginx.conf` :

```
upstream backend {
    server app1:3001;
    server app2:3002;
    server app3:3003;  % Nouvelle instance ajoutee
}
```

   Il faudrait également déployer **app3** dans le `docker-compose.yml` en lui assignant le port **3003**, de la même manière que app1 et app2.